Special Participation E

Using NotebookLM to Infer Learning Priorities and Generate Assessment-Style Questions from Homework Assignments

1. Motivation

Homework assignments are designed to help students practice core concepts, but they do not necessarily reflect how instructors evaluate understanding in exams. While problems in homework often emphasize procedural execution, assessments typically focus on abstraction, conceptual connections, and transfer to novel settings.

In this project, I explore whether an AI-assisted learning tool, NotebookLM, can use a collection of homework assignments to (1) summarize the underlying conceptual priorities of a course, (2) generate assessment-style questions that reflect these priorities, and (3) explicitly link generated questions back to the homework sources that motivate them.

Rather than treating the AI as an oracle for "what will be on the exam", this study critically evaluates whether its inferred priorities and generated questions align with reasonable learning goals, or whether it simply rephrases homework problems.

2. Tool and Data Setup

2.1 Tool Choice: NotebookLM

NotebookLM is a brand-new AI product launched by Google Research and Google Labs. In simple terms, it is an AI learning assistant driven by Google's large model Gemini + an AI knowledge base + a document understanding system.

Its core advantages include:

- ✓ Google ecosystem (Google Drive, Google Docs) is seamlessly integrated.
- ✓ It has strong capabilities for reading PDFs/documents.
- ✓ It focuses on "personalized learning" and research scenarios.
- ✓ The AI will answer based on "your data" rather than guessing through the internet.
- ✓ The output content is highly suitable for academic, research, and content creation.

2.2 Input Materials

The input corpus consists of all homework assignments released for the course

Each assignment is uploaded as a separate source in NotebookLM, preserving document-level provenance.

3. AI Task Design

3.1 AI Task based on prompts

I structured the interaction around three stages, each corresponding to a different learning-oriented capability:

- ✓ Inferring Conceptual Priorities
- ✓ Generating Assessment-Style Questions
- ✓ Linking Questions to Sources

So the prompt is:

You are given a collection of homework assignments from my course.
Use ONLY these documents as sources.

I am a student preparing for the final exam.

Tasks:
1. Based on the homework assignments, identify the topics and skills that are most important for me to review before the final exam.
2. Summarize these exam-relevant concepts clearly and concisely.
3. Generate several final-exam-style practice questions that test understanding and reasoning,
rather than direct repetition of homework problems.
4. For each practice question:
a) Cite which homework assignment(s) motivated it.
b) Briefly explain how the question differs from simply redoing the homework.

Constraints:
- Do NOT replicate or closely mirror homework problems.
- Do NOT introduce material not present or implied in the homework.
- Focus on helping me prepare and assess my understanding, not predicting exact exam questions.

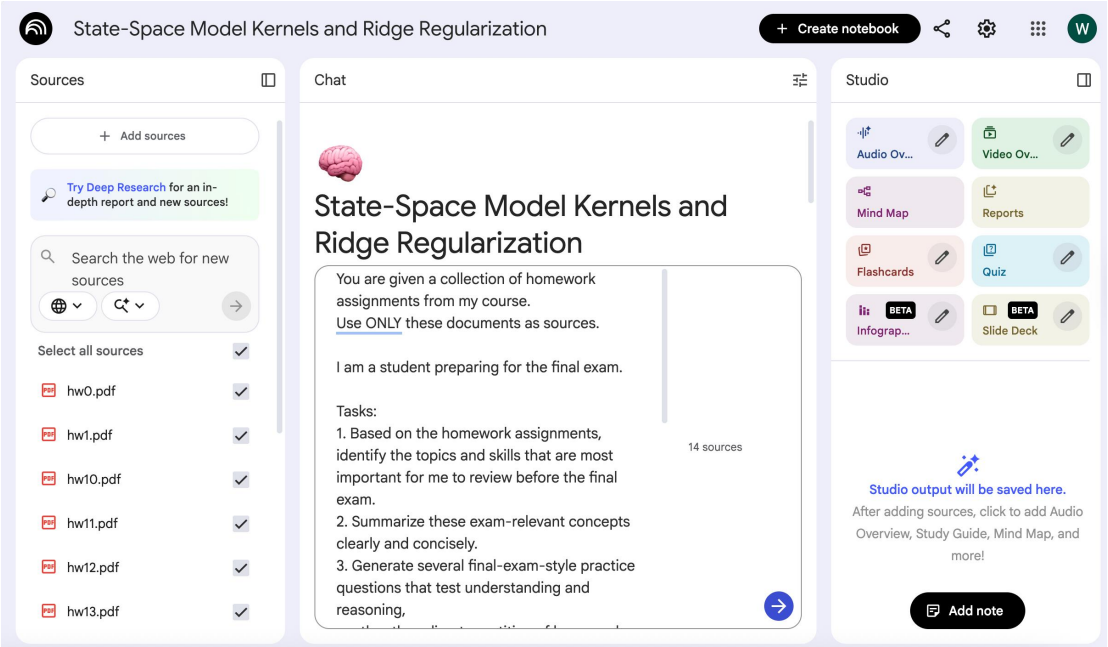
Present the output in clearly separated sections with explicit references to the homework sources.

3.2 Studio provided by NotebookLM

- ✓ Generate a AI mind map based on your sources

✓ Generate a AI interactive quiz based on your sources
✓ Generate a report based on your sources
✓ Generate a AI slide deck based on your sources

4. Interaction Results and Evaluation



This response draws solely on the provided homework assignments (HW0-HW13) to help structure your final exam preparation.

---

1. Important Topics and Skills for Review

The homework assignments cover foundational mathematics, core optimization theory, regularization techniques, major deep learning architectures, and modern topics in scaling, efficiency, and interpretability.

The most important topics and skills, based on their frequent appearance and depth of coverage, are:

Core Area	*Key Topics/Skills*	*Motivating Homeworks*
I. Mathematical & Linear Algebra Foundations	***Vector Calculus/Derivatives:*** *Computing derivatives for quadratic forms and composite functions, understanding dimensions.*	*HW0, HW1, HW12*
	SVD and Matrix Decompositions: *Understanding the role of singular values/eigenvalues in optimization stability, generalization (ridge regression filtering), and architectural analysis (PCA, WOV analysis).*	*HW0, HW1, HW4, HW7, HW8, HW11*
II. Optimization and Convergence	***Gradient Descent Stability:*** *Analyzing convergence for linear systems based on eigenvalues/singular values (σ*	*HW1*

	max ,σmin), setting appropriate learning rates.	
	Advanced Optimizers: *Mechanics and memory/computation trade-offs of SGD with Momentum, Adam, and Muon (Newton-Schulz, SVD scaling).*	*HW1, HW2, HW6, HW11*
	Gradient Estimation: *Policy Gradient vs. Reparameterization Gradient (REINFORCE vs. Pathwise derivatives) for objectives involving expectations over distributions parameterized by θ.*	*HW2, HW3, HW12*
III. Regularization and Normalization	***Ridge Regression/Tikhonov Regularization:*** *Understanding multiple perspectives (Optimization, MAP/Prior, SVD, equivalence to Noise/Dropout) and applications.*	*HW0, HW1, HW5, HW8*
	Batch Normalization (BN): *Purpose, comparison to other normalizations (Layer Norm), computational graph/backpropagation mechanics, behavior in large batch limits.*	*HW5*
IV. Core Architectures and Sequences	***CNN Principles:*** *Dimension tracking, receptive fields, inductive bias, standard vs. depthwise separable convolutions (parameter count reduction).*	*HW4, HW5*
	RNNs: *Forward pass implementation, Vanishing/Exploding Gradients (BPTT context), sequence-to-sequence Encoder-Decoder mechanics.*	*HW7*
	State-Space Models (SSMs): *Convolutional kernel derivation K, exploiting matrix structure (Diagonal, DPLR) for efficient, parallelized computation.*	*HW8*
V. Transformers, Efficiency, and Interpretability	***Softmax Attention:*** *Scaled dot product justification, Multi-Head Attention (MHA) implementation, computational complexity O(N2).*	*HW9*
	Attention Optimization/Efficiency: *Multi-Query Attention (MQA) complexity analysis for incremental decoding, Kernelized Linear Attention (complexity reduction), Ridge Attention (recursive causal computation using Sherman-Morrison).*	*HW8, HW9*
	Architectural Interpretability: *QK Circuit (WQK), OV Circuit (WOV), role of the Residual Stream, SVD analysis of WOV to define Read/Write subspaces.*	*HW11*
VI. Modern Concepts and	***Parameter Efficient Fine-Tuning (PEFT):*** *LoRA (rank k adjustments, memory cost), Soft Prompting (training*	*HW11, HW12*

Scaling	*setup, parameter count, MAML adaptation).*	
	Scaling Laws and Systems: *Chinchilla scaling (N, D, C relationships), estimating memory cost (DRAM, VRAM), computational costs (FLOPs, latency, arithmetic intensity).*	*HW3, HW11*
	Generative Models: *VAEs/VIB (ELBO components, KL regularization effects), Diffusion Models (forward/reverse processes, DDPM/DDIM differentiation).*	*HW12, HW13*
	Alignment: *Direct Preference Optimization (DPO) derivation, bypassing explicit Reward Models and RL loops by leveraging the optimal policy form.*	*HW13*

---

2. Summary of Exam-Relevant Concepts

Optimization, Stability, and Regularization

Gradient Descent Stability: *The convergence rate of Gradient Descent for linear systems is dominated by the ratio of the maximum and minimum singular values ($\sigma max/\sigma min$) of the data matrix X. A scalar learning rate η must be chosen small enough based on $\sigma max2$ to ensure stability.*

Ridge Regression (Tikhonov Regularization): *This technique adds an $\ell 2$ penalty to the weights, $\lambda // w // 22$, to the least squares objective. It can be seen as:*

1. ***SVD Filtering:*** *Attenuating contributions corresponding to small singular values, thereby stabilizing the solution.*

2. ***MAP Estimation:*** *Maximizing the posterior probability given Gaussian noise assumptions on the data and a Gaussian prior on the weights.*

3. ***Equivalent to Noise/Dropout:*** *Adding instance noise to the inputs during training is mathematically equivalent to solving a regularized least squares problem. Similarly, dropout training in linear models is equivalent to Tikhonov regularization.*

Gradient Estimation: *When minimizing an expectation $F(\theta)=Ex\sim p\theta[f(x)]$, if the sampling process is differentiable with respect to θ (e.g., $x=g(z,\theta)$ where $z\sim p(z)$ is noise), the* ***Reparameterization Gradient*** *can be used. If the sampling process is non-differentiable, the*

Policy Gradient (Score Function Estimator) *is used: $\nabla\theta \quad F(\theta)=Ex\sim p\theta[f(x)\nabla\theta \log p\theta(x)]$.*

Maximal Update Parameterization (μP): *This framework defines weight initialization and learning rate scaling to ensure that feature updates (RMS norm squared of output features $y=Wx$) remain constant regardless of the network width, thus preserving feature learning dynamics across different model sizes.*

Architecture: Sequence, Spatial, and Generative Models

CNN Structure: *Convolutional layers introduce spatial weight sharing and translation equivariance, reducing the number of parameters significantly compared to fully connected layers for image data.* ***Depthwise Separable Convolutions*** *further reduce parameters by decoupling spatial filtering (depthwise) from channel mixing (pointwise 1x1 convolution).*

RNN Gradients: *Recurrent networks suffer from* ***Vanishing and Exploding Gradients*** *when the weight matrix Wh in the hidden state update ht =σ(Wh ht−1 +⋯) has singular values significantly smaller or larger than 1, respectively. This is because the gradient information is multiplied by factors of the Jacobian (related to Wh) over many timesteps (Backpropagation Through Time, BPTT).*

State-Space Models (SSMs): *The recurrent SSM xk+1 =Axk +Buk can be reformulated as a convolution yk=∑ℓ Kℓ uk−ℓ where the kernel Kℓ=CAℓB (for ℓ>0 and K0 =D). The convolutional form allows for highly parallelized computation, which is crucial for handling long sequences efficiently, although the recurrent form has a shorter critical path length for generation.*

Generative Models (VAEs & VIB): *The* ***Evidence Lower Bound (ELBO)*** *maximizes the marginal likelihood of the data pθ(x) by combining a reconstruction loss term and a KL divergence term DKL(qϕ(z/x) // p(z)) that forces the learned approximate posterior qϕ to remain close to the prior p(z). In the* ***Variational Information Bottleneck (VIB)****, this KL term acts as a regularization loss to encourage compression.*

Transformers, Efficiency, and Interpretability

Attention Efficiency: *Softmax attention complexity is O(N2D) in computation and O(N2) in memory (where N is sequence length).* ***Multi-Query Attention (MQA)*** *reduces the memory read/write cost during incremental decoding by sharing Key (K) and Value (V) weights across all attention heads.* ***Kernelized Linear Attention*** *replaces the softmax exponential similarity with a kernel and uses the associative property of matrix multiplication to potentially reduce complexity to O(ND2M) or O(NDrandom2 M) if randomized features are used, favorable for very large N.*

Attention Circuits (QK/OV): *In mechanistic interpretability, attention is decomposed into the* ***QK Circuit*** *(determining where to look, defined by WQK =WQT WK) and the* ***OV Circuit*** *(determining what information to move/write, defined by WOV =WO WV). The SVD of WOV reveals the* ***Read Subspace*** *(spanned by V columns) from which information is extracted from source tokens and the* ***Write Subspace*** *(spanned by U columns) to which information is written back to the residual stream.*

Parameter Efficient Fine-Tuning (PEFT):

- ***LoRA (****W=W0 +AB****):*** *Adapts models by freezing the base weight matrix W0 and learning only two low-rank matrices A ∈ Rm×k and B ∈ Rk×l where k is small. The total number of trainable parameters is k(m+l), much smaller than the original matrix size m×l.*
- ***Soft Prompting:*** *Prepends learned vectors (the "soft prompt") to the input sequence embedding, bypassing the token embedding layer and keeping the main model frozen. Training parameters are limited only to the prompt vectors.*

Direct Preference Optimization (DPO): *DPO reformulates the RLHF objective (Max Expected Reward with KL penalty) into a single classification loss based on human preferences. By analyzing the optimal policy $\pi\theta^*$ that solves the constrained maximization, DPO derives an objective that fits the policy directly to the preference data, eliminating the need for an explicit reward model $r\phi$ and the complex RL optimization step.*

---

3. Final Exam Style Practice Questions

Practice Question 1: Convergence and Regularization Synthesis

A researcher attempts to speed up gradient descent on a system $Xw \approx y$ by employing a modified update rule where the optimization step Δw minimizes the linear loss approximation penalized by a generalized Tikhonov term:

$$\Delta w = \arg\Delta w \min \{gT\Delta w + \eta 1\ \Delta wT(XTX + \lambda I)\Delta w\}$$

where $g = \nabla L(w)$ is the gradient of the squared loss, η is the learning rate, and $\lambda \geqslant 0$ is a Tikhonov regularization parameter.

Task: *Determine the stability condition on the learning rate η such that the iterative update $wt+1 = wt + \Delta w$ converges to the optimal solution w^*. Your final answer for the stability range of η should be expressed in terms of the singular values $\{\sigma i\}$ of X and the parameter λ.*

Citation	Difference from Homework
HW1 (Q1), HW0 (Q4, Q6), HW1 (Q6), HW2 (Q1)	This question synthesizes concepts from three distinct areas: (1) stability analysis of gradient descent defined by eigenvalues (HW1), (2) optimization as penalized linear improvement (HW2), and (3) Tikhonov regularization structure (HW0, HW1). It requires finding the closed-form update Δw and then analyzing the recurrence relation, which goes beyond the standard scalar or unregularized GD analysis presented in the sources.

Practice Question 2: Architectural Parameter Efficiency Trade-offs

A large language model researcher is deciding between standard Multi-Head Attention (MHA) layers and Depthwise Separable Convolution (DSC) blocks within a sequence processing module. They plan to use Low-Rank Adaptation (LoRA) for parameter-efficient fine-tuning (PEFT) on both components.

Assume the following dimensions for a single layer/block:

- *Sequence length N=1024.*
- *Hidden dimension D=1024.*
- *MHA: 8 heads, dhead =128.*
- *DSC: Input channels Cin =1024, Output channels Cout =1024. Spatial kernel size K=5.*
- *LoRA: Rank k=16 is used for all weight matrices in both components.*

Task: *Compare the* ***total number of trainable LoRA parameters*** *required for one layer of MHA versus one DSC block (Depthwise + Pointwise steps). Explain which architectural choice (MHA or DSC) provides a greater relative reduction in fine-tuning parameters compared to its original full-rank implementation, and briefly explain why based on the structure of the two blocks. Ignore biases and Layer Norm weights.*

Citation	*Difference from Homework*
HW11 (Q1), HW5 (Q3), HW9 (Q4)	*This question requires integrating parameter counting skills for two distinct architectures (DSC and MHA) and applying the complexity analysis of LoRA across both. While HW5 analyzes DSC complexity and HW11 analyzes LoRA memory, this task forces a direct quantitative comparison of parameter savings efficiency across different network structures.*

Practice Question 3: Generative Model Gradient Flow

The training of a Variational Autoencoder (VAE) involves minimizing the negative ELBO, which contains a reconstruction term $E_{z\sim q_\phi(z|x)}[-\log p_\theta(x|z)]$ *and a KL term* $D_{KL}(q_\phi(z|x) \| p(z))$*. The encoder* $q_\phi(z|x)$ *is modeled as a Gaussian* $N(\mu_\phi(x), \Sigma_\phi(x))$*, where* μ *and* Σ *are outputs of a neural network parameterized by* ϕ*.*

When computing the gradient ∇_ϕ *with respect to the encoder parameters* ϕ*:*

Task:

1. Explain why the Reparameterization Trick is essential for calculating the gradient of the reconstruction loss term and flowing updates back to ϕ*.*

2. If the training objective $F(\phi)$ *only included the KL regularization term,* $D_{KL}(q_\phi(z|x) \| p(z))$*, would the Reparameterization Trick still be strictly necessary to compute* $\nabla_\phi F(\phi)$*? Justify your answer by referencing how gradients are typically computed for KL divergence between two Gaussians in the context of VAEs/VIB, and explain whether the term involves non-differentiable sampling based on* ϕ*.*

Citation	*Difference from Homework*
HW12 (Q4), HW3 (Q4), HW12 (Q3)	*This tests the fundamental conceptual difference between the structure of the reconstruction term (where the expectation is critical to differentiate through) and the KL divergence term, which often has a closed-form derivative (as implied by VIB analysis where the KL term contributes to* $\nabla_\theta e$*). The distinction requires assessing where the dependence on z (the sampled variable) lies versus where the explicit dependence on* ϕ *(the parameters) lies.*

Practice Question 4: Interpretability and PEFT Interaction

Consider a single layer of a Transformer decoder where LoRA is applied only to the output weight matrix W_O, such that $W_O = W_{O,0} + AB$, where $W_{O,0}$ is frozen and A,B are learned low-rank matrices. All other matrices (W_Q, W_K, W_V) are frozen at their pre-trained values.

Recall that the information movement capacity of an attention head is governed by the $W_{OV} = W_O W_V$ matrix, whose SVD $W_{OV} = U\Sigma V^T$ defines the read and write subspaces.

Task:

1. Derive the expression for the effective WOV matrix in terms of $W_{O,0}$, A, B, and W_V

2. Explain how applying LoRA only to WO limits the capacity of the head to learn new representations. Specifically, which subspace—the Read Subspace or the Write Subspace—remains entirely constrained by the pre-trained model $W_{O,0}$ (i.e., cannot change during fine-tuning), and which subspace gains new, limited flexibility from the AB term?

Citation	Difference from Homework
HW11 (Q1, Q2e), HW12 (Q7)	This links the structural analysis of the OV circuit (HW11), which analyzes subspaces via SVD, directly to the mechanism of parameter efficient fine-tuning (LoRA). It requires synthesizing the mathematical construction of LoRA into the composite WOV matrix and analyzing how that composition affects the definition of the read (V) and write (U) subspaces, testing conceptual understanding rather than rote calculation.

Practice Question 5: Alignment and Reward Modeling

In the standard RLHF pipeline, the Reward Model (RM) $r_\phi(x,y)$ is trained using the negative log-likelihood loss derived from the Bradley-Terry (BT) model.

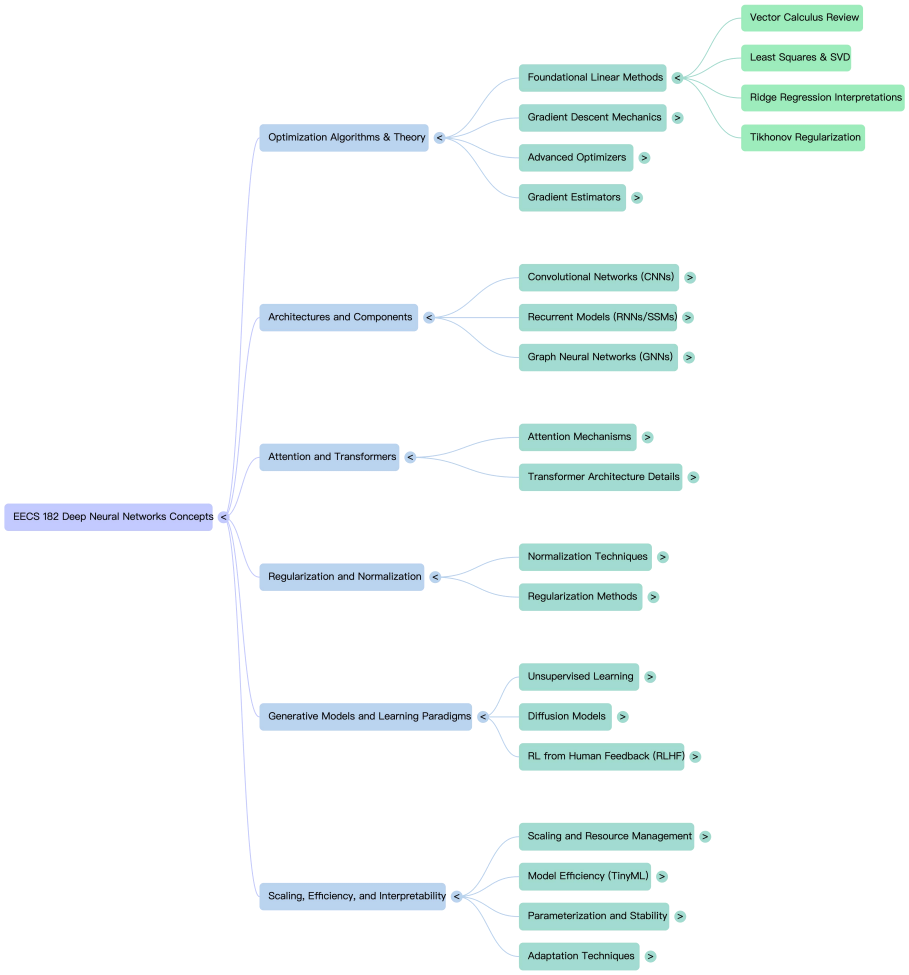
Task:

1. Assume a scenario where the RM is perfectly trained using the BT model loss, but the subsequent RL fine-tuning phase (using PPO) is skipped entirely, leaving only the SFT policy π_{SFT}. Explain why, despite having a perfect RM, the SFT policy π_{SFT} might fail to generate high-quality responses that maximize the true reward $r^*(x,y)$.

2. Contrast the necessity of having a learned proxy reward function $r_\phi(x,y)$ in the RLHF pipeline versus the Direct Preference Optimization (DPO) objective L_{DPO}. Explain the implicit role that the log probability ratio $\log \pi_{ref}(y \mid x) \pi_\theta(y \mid x)$ plays in L_{DPO} that eliminates the need for $r_\phi(x,y)$.

Citation	Difference from Homework
HW13 (Q2)	This question directly probes the underlying weakness of the three-stage RLHF process (SFT → RM → RL), requiring the student to understand why the

PPO/RL step is needed, even if the RM is perfect. It then requires conceptual insight into how DPO implicitly models the reward difference using the policy ratio, synthesizing the full derivation of DPO into a single explanation of its theoretical advantage.



Mind Map generated by NotebookLM

5. Critical Analysis

From the perspective of a student preparing for a final exam, this output is highly structured, comprehensive, and academically demanding. It demonstrates a strong ability to synthesize concepts across multiple homework assignments rather than focusing on isolated problems. However, its usefulness is uneven: topic identification and conceptual summaries are particularly strong, while some generated practice

questions approach the boundary between genuine conceptual transfer and advanced variants of homework problems.

5.1 Important Topics and Concepts for Review

Strengths:

- ✓ The breadth of coverage is excellent, spanning mathematics, optimization, architectures, generative models, efficiency, and alignment.
- ✓ Explicitly linking each topic to motivating homework assignments is highly beneficial, as it allows the student to trace abstract themes back to concrete practice.
- ✓ Grouping topics into high-level core areas provides a clear mental map of the course.

Limitations:

- ✓ Some conceptual boundaries blur across categories (e.g., optimization, regularization, and scaling), which may hide higher-level unifying principles.

5.2 Final Exam Style Practice Questions

- ✓ Question 1 effectively synthesizes optimization stability and regularization concepts. However, the structure is close enough to advanced homework problems that it may feel like a generalized HW question with increased complexity.
- ✓ Question 3 is one of the strongest questions. It cleanly tests conceptual understanding of gradient flow and reparameterization without procedural overload.
- ✓ Question 5 probes conceptual understanding of RLHF vs. DPO. It is clearly not a homework copy and encourages theoretical reasoning.

6. Learning Implications

From a learning perspective, NotebookLM's core functions includes:

- ✓ Document upload (PDF, PPT, text, Excel)
- ✓ AI summary generation
- ✓ Cross-summary of multiple documents
- ✓ Automatic generation of structured notes
- ✓ "Ask Anything" document understanding
- ✓ Automatic synchronization output of Google Docs
- ✓ Organization of multi-document knowledge base Notebook

It is suitable for the scenarios like:

- ✓ Summary of the paper
- ✓ Exam review (automatically generating an outline)
- ✓ Industry report organization

- ✓ Content production
- ✓ Reading notes
- ✓ Multi-document comparison analysis
- ✓ Project research

7. Executive Summary

Most questions avoid direct replication. As a final exam preparation tool, NotebookLM is best used as a synthesis and challenge generator rather than a strict study guide. It excels at revealing conceptual connections across assignments but should be complemented with personal notes and prior mistakes.

NotebookLM also supports mind mapping, quizzes, generating slides or reports, which are very useful for exam preparation. Additionally, I observed that NotebookLM also provides understanding of videos and audio. This is closely related to my previous project exploring the illusions of large language models in video understanding tasks of knowledge types. I originally intended to test the long video understanding using the classroom videos of EECS182, but due to permission issues, this could not be done.

I believe that for final review, students can fully utilize the tools provided by NotebookLM, as it covers almost all modal inputs and outputs in the knowledge learning process of students.